Neural networks as composed affine and spectral operators

MATH 2740 – Mathematics of Data Science – Unit U12

Julien Arino
julien.arino@umanitoba.ca

Department of Mathematics @ University of Manitoba

Fall 2026

The University of Manitoba campuses are located on original lands of Anishinaabeg, Ininew, Anisininew, Dakota and Dene peoples, and on the National Homeland of the Red River Métis. We respect the Treaties that were made on these territories, we acknowledge the harms and mistakes of the past, and we dedicate ourselves to move forward in partnership with Indigenous communities in a spirit of Reconciliation and collaboration.

Outline

From single neurons to affine matrix layers

Why non-linearity is essential: collapsing linear networks

Linear autoencoders and subspace projections (bridge to PCA)

Graph convolutional networks: combining left and right matrix multiplication

Why neural networks belong in a linear algebra course

Modern machine learning and artificial intelligence are often presented through the lens of multivariable calculus and black-box software libraries

At their structural core, however, neural networks are **compositions of matrix operators** interleaved with simple coordinate-wise non-linearities

In this unit, we unify the major threads of the course:

- **Affine maps and hyperplanes** (Unit 06): a single neuron is an affine functional followed by a threshold or activation function
- **Matrix multiplication and rank bounds**: forward propagation is batch matrix multiplication and non-linearity prevents rank collapse
- **SVD and PCA** (Units 04 and 05): linear autoencoders learn orthogonal projections onto principal subspaces
- **Graph matrices and Markov chains** (Units 08 to 10): graph convolutional networks combine left-multiplication by normalised adjacency matrices with right-multiplication by weight matrices

From single neurons to affine matrix layers

Why non-linearity is essential: collapsing linear networks

Linear autoencoders and subspace projections (bridge to PCA)

Graph convolutional networks: combining left and right matrix multiplication

From single neurons to affine matrix layers

Single artificial neuron and the perceptron

Multi-layer feedforward networks (MLPs) in vector and batch matrix form

Biological inspiration and the McCulloch-Pitts-Rosenblatt neuron

Artificial neural networks *(ANNs) are computing systems inspired by biological neural networks. A collection of connected nodes called* ***artificial neurons*** *receive real-valued signals along weighted edges (synapses), aggregate them via an affine combination and emit an output through a non-linear* ***activation function***

- McCulloch and Pitts (1943) introduced threshold logic units for Boolean computation
- Rosenblatt (1957) introduced the **perceptron** for pattern classification (such as recognising letters or digits from a pixel grid called the *retina*)
- Mathematically, a single artificial neuron is a direct generalisation of the linear classifiers and separating hyperplanes studied in Unit 06

The single artificial neuron as an affine functional

Definition 180 (Single artificial neuron)

Given an input feature vector $\boldsymbol{x} = (x_1, \ldots, x_p)^T \in \mathbb{R}^p$*, a* ***single artificial neuron*** *is parameterised by a* ***weight vector*** $\boldsymbol{w} = (w_1, \ldots, w_p)^T \in \mathbb{R}^p$ *and a scalar* ***bias*** $b \in \mathbb{R}$*. It computes:*

1. *An* ***affine pre-activation*** *(or* ***logit****):*

$$z = \boldsymbol{w}^T \boldsymbol{x} + b = \sum_{j=1}^{p} w_j x_j + b \in \mathbb{R}$$

2. *A scalar* ***activation*** *(output):*

$$a = \sigma(z) = \sigma\left(\boldsymbol{w}^T \boldsymbol{x} + b\right) \in \mathbb{R}$$

where $\sigma : \mathbb{R} \to \mathbb{R}$ *is a chosen* ***activation function***

Absorbing the bias via augmented coordinates

In classical perceptron notation, a threshold θ is used so the neuron fires when $\boldsymbol{w}^T\boldsymbol{x} > \theta$; setting $b = -\theta$ gives $\boldsymbol{w}^T\boldsymbol{x} + b > 0$

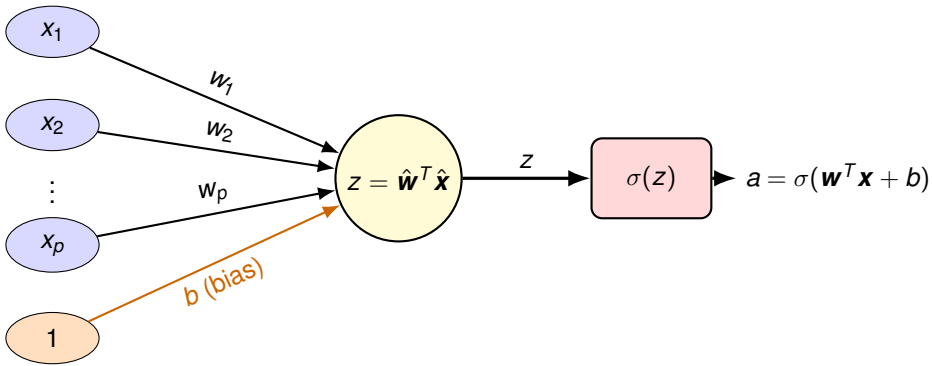
Just as in homogeneous coordinates, we can absorb the bias b into the weight vector by introducing a constant "dummy" input feature $x_0 = 1$:

$$\hat{\boldsymbol{x}} = \begin{pmatrix} \boldsymbol{x} \\ 1 \end{pmatrix} = \begin{pmatrix} x_1 \\ \vdots \\ x_p \\ 1 \end{pmatrix} \in \mathbb{R}^{p+1}, \qquad \hat{\boldsymbol{w}} = \begin{pmatrix} \boldsymbol{w} \\ b \end{pmatrix} = \begin{pmatrix} w_1 \\ \vdots \\ w_p \\ b \end{pmatrix} \in \mathbb{R}^{p+1}$$

Then the affine pre-activation becomes a pure inner product in $\mathbb{R}^{p+1}$:

$$z = \boldsymbol{w}^T\boldsymbol{x} + b = \hat{\boldsymbol{w}}^T\hat{\boldsymbol{x}}$$

Diagram of a single neuron with augmented bias input



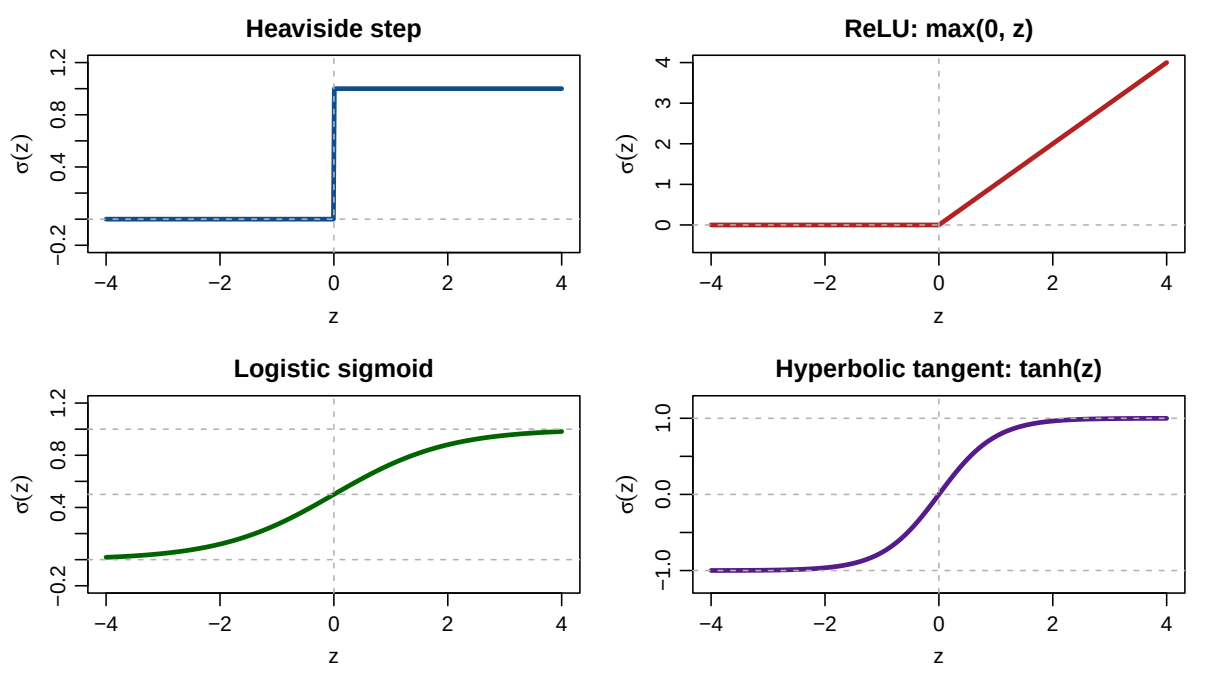
Geometrically, the locus $\{\boldsymbol{x} \in \mathbb{R}^p : \boldsymbol{w}^T \boldsymbol{x} + b = 0\}$ is an **affine hyperplane** with normal vector $\boldsymbol{w}$, partitioning $\mathbb{R}^p$ into two half-spaces $\{\boldsymbol{w}^T \boldsymbol{x} + b > 0\}$ and $\{\boldsymbol{w}^T \boldsymbol{x} + b < 0\}$

Common scalar activation functions

Definition 181 (Component-wise activation functions)

When applied to a vector $\boldsymbol{z} \in \mathbb{R}^p$ or matrix $Z \in \mathcal{M}_{np}$, a scalar activation function $\sigma : \mathbb{R} \to \mathbb{R}$ acts **entrywise**: $\left[\sigma(Z)\right]_{ij} = \sigma(z_{ij})$. Standard choices include:

- **Heaviside step (threshold)**: $H(z) = \begin{cases} 0 & \text{if } z \leq 0 \\ 1 & \text{if } z > 0 \end{cases}$
- **Rectified linear unit (ReLU)**: $\mathrm{ReLU}(z) = \max(0, z) = \begin{cases} 0 & \text{if } z \leq 0 \\ z & \text{if } z > 0 \end{cases}$
- **Logistic sigmoid**: $\sigma_{\mathrm{sig}}(z) = \dfrac{1}{1 + e^{-z}} \in (0, 1)$
- **Hyperbolic tangent**: $\tanh(z) = \dfrac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma_{\mathrm{sig}}(2z) - 1 \in (-1, 1)$


Heaviside step
σ(z)
z
ReLU: max(0, z)
σ(z)
z
Logistic sigmoid
σ(z)
z
Hyperbolic tangent: tanh(z)
σ(z)
z

Single-layer perceptrons on Boolean logic gates: OR and AND

Consider binary inputs $\boldsymbol{x} = (x_1, x_2)^T \in \{0, 1\}^2$ with the Heaviside step activation $H(z)$

Logical OR gate:

x_1	x_2	$z = x_1 + x_2 - 0.5$	$H(z)$
0	0	-0.5	0
1	0	$+0.5$	1
0	1	$+0.5$	1
1	1	$+1.5$	1

Solved by $\boldsymbol{w} = (1, 1)^T$ and $b = -0.5$

Logical AND gate:

x_1	x_2	$z = x_1 + x_2 - 1.5$	$H(z)$
0	0	-1.5	0
1	0	-0.5	0
0	1	-0.5	0
1	1	$+0.5$	1

Solved by $\boldsymbol{w} = (1, 1)^T$ and $b = -1.5$

In both cases, a single line $x_1 + x_2 + b = 0$ separates the class-0 points from the class-1 points in $\mathbb{R}^2$

The perceptron and Widrow-Hoff learning rule in vector form

How does a single neuron learn its augmented weight vector $\hat{\boldsymbol{w}} = (\boldsymbol{w}^T, b)^T \in \mathbb{R}^{p+1}$ from labelled training pairs $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$ with $y_i \in \{0, 1\}$?

In **supervised error-correction learning** (Widrow-Hoff / delta rule), when input $\hat{\boldsymbol{x}}_i = (\boldsymbol{x}_i^T, 1)^T$ produces output $\hat{y}_i = H\big((\hat{\boldsymbol{w}}^{(t)})^T \hat{\boldsymbol{x}}_i\big)$, we update the weight vector by

$$\hat{\boldsymbol{w}}^{(t+1)} = \hat{\boldsymbol{w}}^{(t)} + \eta\big(y_i - \hat{y}_i\big)\hat{\boldsymbol{x}}_i$$

where $\eta > 0$ is the **learning rate**

Geometric meaning via inner products:

- If $\hat{y}_i = y_i$ (correct prediction), $\hat{\boldsymbol{w}}^{(t+1)} = \hat{\boldsymbol{w}}^{(t)}$ (no change)
- If $y_i = 1$ but $\hat{y}_i = 0$ (false negative), $\hat{\boldsymbol{w}}^{(t+1)} = \hat{\boldsymbol{w}}^{(t)} + \eta\hat{\boldsymbol{x}}_i$, so the new pre-activation on $\hat{\boldsymbol{x}}_i$ strictly increases:

$$\big(\hat{\boldsymbol{w}}^{(t+1)}\big)^T \hat{\boldsymbol{x}}_i = \big(\hat{\boldsymbol{w}}^{(t)}\big)^T \hat{\boldsymbol{x}}_i + \eta\|\hat{\boldsymbol{x}}_i\|_2^2 > \big(\hat{\boldsymbol{w}}^{(t)}\big)^T \hat{\boldsymbol{x}}_i$$

tilting the hyperplane normal $\hat{\boldsymbol{w}}$ towards $\hat{\boldsymbol{x}}_i$!

Worked trace: two perceptron updates to learn OR

Let us trace the update rule $\hat{\boldsymbol{w}}^{(t+1)} = \hat{\boldsymbol{w}}^{(t)} + \eta(y_i - \hat{y}_i)\hat{\boldsymbol{x}}_i$ with $\eta = 0.1$ and augmented inputs $\hat{\boldsymbol{x}} = (x_1, x_2, 1)^T$, starting from $\hat{\boldsymbol{w}}^{(0)} = (0, 0, 0)^T$:

1. **Step 1**: Present $\boldsymbol{x} = (1, 1)^T$, so $\hat{\boldsymbol{x}} = (1, 1, 1)^T$ with target $y = 1$. Since $\left(\hat{\boldsymbol{w}}^{(0)}\right)^T \hat{\boldsymbol{x}} = 0 \leq 0$, we predict $\hat{y} = 0$ (error $y - \hat{y} = 1$):

$$\hat{\boldsymbol{w}}^{(1)} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} + 0.1(1 - 0) \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0.1 \\ 0.1 \\ 0.1 \end{pmatrix}$$

2. **Step 2**: Present $\boldsymbol{x} = (0, 0)^T$, so $\hat{\boldsymbol{x}} = (0, 0, 1)^T$ with target $y = 0$. Since $\left(\hat{\boldsymbol{w}}^{(1)}\right)^T \hat{\boldsymbol{x}} = 0.1 > 0$, we predict $\hat{y} = 1$ (error $y - \hat{y} = -1$):

$$\hat{\boldsymbol{w}}^{(2)} = \begin{pmatrix} 0.1 \\ 0.1 \\ 0.1 \end{pmatrix} + 0.1(0 - 1) \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0.1 \\ 0.1 \\ 0 \end{pmatrix}$$

With $\boldsymbol{w} = (0.1, 0.1)^T$ and $b = 0$, $H(0.1x_1 + 0.1x_2)$ classifies all four OR inputs correctly!

Why a single neuron fails on XOR (Minsky and Papert, 1969)

Now consider the **exclusive OR (XOR)** truth table on $\{0,1\}^2$:

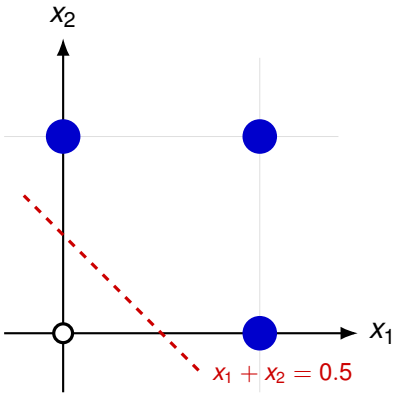
$$(0,0) \mapsto 0, \qquad (1,0) \mapsto 1, \qquad (0,1) \mapsto 1, \qquad (1,1) \mapsto 0$$

Suppose a single neuron with weights $\boldsymbol{w} = (w_1, w_2)^T$ and bias b could represent XOR via $H(\boldsymbol{w}^T \boldsymbol{x} + b)$. Then the four inputs require:

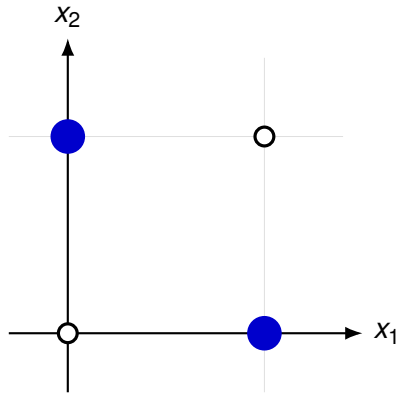
$$\begin{aligned}
(0,0) \mapsto 0 &\implies b \leq 0 \\
(1,0) \mapsto 1 &\implies w_1 + b > 0 \\
(0,1) \mapsto 1 &\implies w_2 + b > 0 \\
(1,1) \mapsto 0 &\implies w_1 + w_2 + b \leq 0
\end{aligned}$$

Adding the second and third inequalities gives $w_1 + w_2 + 2b > 0$. Since $b \leq 0$, this implies $w_1 + w_2 + b \geq w_1 + w_2 + 2b > 0$, contradicting $w_1 + w_2 + b \leq 0$!

Linear separability: OR vs XOR in $\mathbb{R}^2$



OR: linearly separable



XOR: not linearly separable

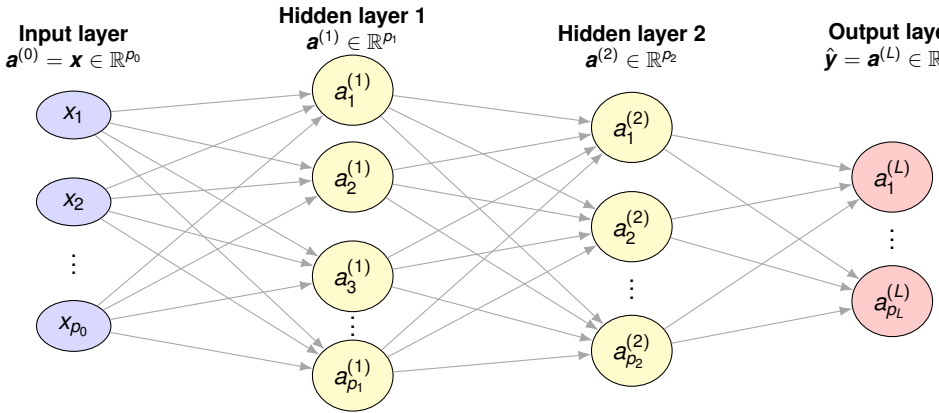
To solve non-linearly separable problems, we must **compose** multiple layers: a hidden layer first maps the inputs into a new coordinate space where they become linearly separable

From single neurons to affine matrix layers

Single artificial neuron and the perceptron

Multi-layer feedforward networks (MLPs) in vector and batch matrix form

Architecture of a multi-layer feedforward network (MLP)



Single-vector formulation of an MLP layer

Definition 182 (Affine neural network layer – vector form)

An L-layer ***multi-layer perceptron (MLP)*** *has layer widths* $p_0, p_1, \ldots, p_L \in \mathbb{N}_+$ *(where* $p_0 = p$ *is the number of input features). Given a single input column vector* $\boldsymbol{a}^{(0)} = \boldsymbol{x} \in \mathbb{R}^{p_0}$*, each layer* $\ell \in \{1, \ldots, L\}$ *maps* $\boldsymbol{a}^{(\ell-1)} \in \mathbb{R}^{p_{\ell-1}}$ *to* $\boldsymbol{a}^{(\ell)} \in \mathbb{R}^{p_\ell}$ *via*

$$\boldsymbol{z}^{(\ell)} = W^{(\ell)} \boldsymbol{a}^{(\ell-1)} + \boldsymbol{b}^{(\ell)} \in \mathbb{R}^{p_\ell}, \qquad \boldsymbol{a}^{(\ell)} = \sigma_\ell\big(\boldsymbol{z}^{(\ell)}\big) \in \mathbb{R}^{p_\ell}$$

where:

- $W^{(\ell)} = \big[w_{ij}^{(\ell)}\big] \in \mathcal{M}_{p_\ell, p_{\ell-1}}$ *is the* ***weight matrix*** *of layer* ℓ *(*p_ℓ *rows for outputs,* $p_{\ell-1}$ *columns for inputs)*
- $\boldsymbol{b}^{(\ell)} \in \mathbb{R}^{p_\ell}$ *is the* ***bias vector*** *of layer* ℓ
- $\sigma_\ell : \mathbb{R}^{p_\ell} \to \mathbb{R}^{p_\ell}$ *is the component-wise activation function of layer* ℓ

From single vectors to batch matrices

In data science (Units 04 and 05), a dataset of n observations with $p_0 = p$ features is stored as a data matrix $X \in \mathcal{M}_{n,p_0}$ whose **rows** are transposed observation vectors (in deep learning, a subset of n observations is called a **batch** or **mini-batch**, often denoted B, and X is called the **batch input matrix**):

$$X = \begin{pmatrix} \boldsymbol{x}_1^T \\ \boldsymbol{x}_2^T \\ \vdots \\ \boldsymbol{x}_n^T \end{pmatrix} \in \mathcal{M}_{n,p_0}$$

For observation $i \in \{1, \ldots, n\}$, the column vector pre-activation in layer 1 is $\boldsymbol{z}_i^{(1)} = W^{(1)}\boldsymbol{x}_i + \boldsymbol{b}^{(1)} \in \mathbb{R}^{p_1}$. Taking transposes to store outputs as **rows** gives:

$$\left(\boldsymbol{z}_i^{(1)}\right)^T = \boldsymbol{x}_i^T \left(W^{(1)}\right)^T + \left(\boldsymbol{b}^{(1)}\right)^T \in \mathcal{M}_{1,p_1}$$

How do we stack all n rows simultaneously into a single matrix equation?

Batch matrix formulation and rank-1 bias broadcasting

Definition 183 (Batch matrix forward propagation)

Given a data matrix (batch input matrix) $A^{(0)} = X \in \mathcal{M}_{n,p_0}$ whose n rows are input observations, layer $\ell \in \{1, \ldots, L\}$ computes the pre-activation matrix $Z^{(\ell)} \in \mathcal{M}_{n,p_\ell}$ and activation matrix $A^{(\ell)} \in \mathcal{M}_{n,p_\ell}$ via

$$Z^{(\ell)} = A^{(\ell-1)}\left(W^{(\ell)}\right)^T + \mathbb{1}_n\left(\boldsymbol{b}^{(\ell)}\right)^T \in \mathcal{M}_{n,p_\ell}, \qquad A^{(\ell)} = \sigma_\ell\left(Z^{(\ell)}\right) \in \mathcal{M}_{n,p_\ell}$$

where $\mathbb{1}_n = (1, 1, \ldots, 1)^T \in \mathbb{R}^n$ is the all-ones column vector

Why does $\mathbb{1}_n\left(\boldsymbol{b}^{(\ell)}\right)^T$ appear? Because the outer product of $\mathbb{1}_n \in \mathcal{M}_{n1}$ and $\left(\boldsymbol{b}^{(\ell)}\right)^T \in \mathcal{M}_{1,p_\ell}$ is a **rank-1 matrix** that replicates the bias row vector across all n observations:

$$\mathbb{1}_n\left(\boldsymbol{b}^{(\ell)}\right)^T = \begin{pmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix} \begin{pmatrix} b_1^{(\ell)} & b_2^{(\ell)} & \cdots & b_{p_\ell}^{(\ell)} \end{pmatrix} = \begin{pmatrix} b_1^{(\ell)} & b_2^{(\ell)} & \cdots & b_{p_\ell}^{(\ell)} \\ b_1^{(\ell)} & b_2^{(\ell)} & \cdots & b_{p_\ell}^{(\ell)} \\ \vdots & \vdots & \ddots & \vdots \\ b_1^{(\ell)} & b_2^{(\ell)} & & b_{p_\ell}^{(\ell)} \end{pmatrix} \in \mathcal{M}_{n,p_\ell}$$

Block matrix view: absorbing the bias column into $A^{(\ell-1)}$

Just as for a single vector, we can also express the batch affine transformation as a single block matrix product by appending a column of ones $\mathbb{1}_n$ to $A^{(\ell-1)}$ and the bias column $\boldsymbol{b}^{(\ell)}$ to $W^{(\ell)}$:

$$\hat{A}^{(\ell-1)} = \left[\, A^{(\ell-1)} \;\middle|\; \mathbb{1}_n \,\right] \in \mathcal{M}_{n,p_{\ell-1}+1}, \qquad \hat{W}^{(\ell)} = \left[\, W^{(\ell)} \;\middle|\; \boldsymbol{b}^{(\ell)} \,\right] \in \mathcal{M}_{p_\ell,p_{\ell-1}+1}$$

By block matrix multiplication,

$$\hat{A}^{(\ell-1)}\big(\hat{W}^{(\ell)}\big)^T = \left[\, A^{(\ell-1)} \;\middle|\; \mathbb{1}_n \,\right] \begin{bmatrix} \big(W^{(\ell)}\big)^T \\ \big(\boldsymbol{b}^{(\ell)}\big)^T \end{bmatrix} = A^{(\ell-1)}\big(W^{(\ell)}\big)^T + \mathbb{1}_n\big(\boldsymbol{b}^{(\ell)}\big)^T = Z^{(\ell)}$$

Both views are algebraically identical: the rank-1 outer product $\mathbb{1}_n\big(\boldsymbol{b}^{(\ell)}\big)^T$ is simply the contribution of the constant column $\mathbb{1}_n$!

Dimension tracking and parameter counting

In software (such as R or PyTorch), shape mismatches are the most common source of bugs. Tracking dimensions systematically is essential

Proposition 184 (Dimension and parameter count of an L-layer MLP)

For a batch of n observations passing through an MLP with widths $p_0, p_1, \ldots, p_L$:

- *At layer $\ell \in \{1, \ldots, L\}$:*

$$\underbrace{A^{(\ell-1)}}_{n \times p_{\ell-1}} \underbrace{\left(W^{(\ell)}\right)^T}_{p_{\ell-1} \times p_\ell} + \underbrace{\mathbb{1}_n}_{n \times 1} \underbrace{\left(\boldsymbol{b}^{(\ell)}\right)^T}_{1 \times p_\ell} = \underbrace{Z^{(\ell)}}_{n \times p_\ell} \xrightarrow{\sigma_\ell} \underbrace{A^{(\ell)}}_{n \times p_\ell}$$

- *The total number of trainable scalar parameters (weights and biases) is*

$$P = \sum_{\ell=1}^{L} \left(p_\ell p_{\ell-1} + p_\ell\right) = \sum_{\ell=1}^{L} p_\ell \left(p_{\ell-1} + 1\right)$$

*Notice that P is **independent of the number of observations** n*

Example: dimension tracking and parameter counting

Problem: Consider a 3-layer MLP with input dimension $p_0 = 4$, hidden widths $p_1 = 5$ and $p_2 = 3$ and output dimension $p_3 = 2$, evaluated on a batch of $n = 10$ observations. Track all matrix dimensions and count the total number of trainable parameters

Layer ℓ	$A^{(\ell-1)}$	$W^{(\ell)}$	$\left(W^{(\ell)}\right)^T$	$\boldsymbol{b}^{(\ell)}$	$\mathbb{1}_n\left(\boldsymbol{b}^{(\ell)}\right)^T$	$Z^{(\ell)}, A^{(\ell)}$
$\ell = 1$	10×4	5×4	4×5	5×1	10×5	10×5
$\ell = 2$	10×5	3×5	5×3	3×1	10×3	10×3
$\ell = 3$	10×3	2×3	3×2	2×1	10×2	10×2

Total trainable scalar parameters:

- Layer 1: $p_1 p_0 + p_1 = 5 \times 4 + 5 = 25$
- Layer 2: $p_2 p_1 + p_2 = 3 \times 5 + 3 = 18$
- Layer 3: $p_3 p_2 + p_3 = 2 \times 3 + 2 = 8$
- **Total:** $P = 25 + 18 + 8 = 51$ parameters

Example by hand: solving XOR with a 2-layer ReLU network

Problem: Evaluate a 2-layer network ($p_0 = 2, p_1 = 2, p_2 = 1$) on all $n = 4$ points of $\{0,1\}^2$ simultaneously, with ReLU hidden activation $\sigma_1(z) = \max(0, z)$ and linear output $\sigma_2(z) = z$:

$$X = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix} \in \mathcal{M}_{42}, \quad W^{(1)} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \in \mathcal{M}_{22}, \quad \boldsymbol{b}^{(1)} = \begin{pmatrix} 0 \\ -1 \end{pmatrix} \in \mathbb{R}^2$$

$$W^{(2)} = \left(\boldsymbol{w}^{(2)}\right)^T = \begin{pmatrix} 1 & -2 \end{pmatrix} \in \mathcal{M}_{12}, \quad b^{(2)} = 0 \in \mathbb{R}$$

Step 1 – Hidden pre-activations $Z^{(1)} = X\left(W^{(1)}\right)^T + \mathbb{1}_4\left(\boldsymbol{b}^{(1)}\right)^T$:

$$Z^{(1)} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 0 & -1 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 2 & 2 \end{pmatrix} + \begin{pmatrix} 0 & -1 \\ 0 & -1 \\ 0 & -1 \\ 0 & -1 \end{pmatrix} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix}$$

Example by hand: hidden activations and output layer

Step 2 – Apply ReLU entrywise $A^{(1)} = \max\left(0, Z^{(1)}\right)$:

$$A^{(1)} = \max\left(0, \begin{pmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix}\right) = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix} \in \mathcal{M}_{42}$$

Step 3 – Output layer $\hat{\boldsymbol{y}} = Z^{(2)} = A^{(1)}\left(W^{(2)}\right)^T + \mathbb{1}_4 b^{(2)}$:

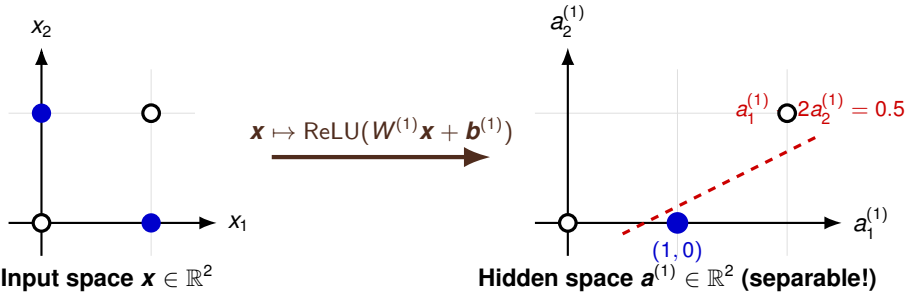
$$\hat{\boldsymbol{y}} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ -2 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0(1) + 0(-2) \\ 1(1) + 0(-2) \\ 1(1) + 0(-2) \\ 2(1) + 1(-2) \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$$

The 2-layer ReLU network reproduces the exact XOR truth table on all four points!

Geometric interpretation: how the hidden layer untangles XOR

Look at the rows of the hidden activation matrix $A^{(1)}$ as coordinates in the hidden space $\mathbb{R}^2$:

$$\boldsymbol{x}_1 = (0,0)^T \mapsto \boldsymbol{a}_1^{(1)} = (0,0)^T, \quad \boldsymbol{x}_2 = (1,0)^T \text{ and } \boldsymbol{x}_3 = (0,1)^T \mapsto (1,0)^T, \quad \boldsymbol{x}_4 = (1,1)^T \mapsto$$



Verification in R: batch matrix forward pass for XOR

```
X = matrix(c(0, 0,  1, 0,  0, 1,  1, 1), nrow = 4, byrow = TRUE)
W1 = matrix(c(1, 1,  1, 1), nrow = 2, byrow = TRUE)
b1 = c(0, -1);  one_n = rep(1, nrow(X))
Z1 = X %*% t(W1) + one_n %*% t(b1)
A1 = pmax(0, Z1)
y_hat = A1 %*% c(1, -2)

## Error in `A1 %*% c(1, -2)`:
## !  non-conformable arguments

cbind(X, A1 = A1, y_hat = as.vector(y_hat))

## Error:
## !  object 'y_hat' not found
```

From single neurons to affine matrix layers

Why non-linearity is essential: collapsing linear networks

Linear autoencoders and subspace projections (bridge to PCA)

Graph convolutional networks: combining left and right matrix multiplication

What if all activation functions are linear?

Suppose we build a deep neural network with L layers, but we omit the non-linear activations by setting every activation function to the identity map:

$$\sigma_\ell(\boldsymbol{z}) = \boldsymbol{z} \qquad \forall \ell \in \{1, \ldots, L\}$$

Such an architecture is called a **deep linear network**

At each layer ℓ, the transformation is purely affine:

$$\boldsymbol{a}^{(\ell)} = W^{(\ell)} \boldsymbol{a}^{(\ell-1)} + \boldsymbol{b}^{(\ell)}$$

What is the expressive power of composing L affine transformations in sequence?

Collapse of composed affine maps

Theorem 185 (Collapse of deep linear networks)

*Let $f_\ell(\boldsymbol{u}) = W^{(\ell)}\boldsymbol{u} + \boldsymbol{b}^{(\ell)}$ be affine maps with $W^{(\ell)} \in \mathcal{M}_{p_\ell, p_{\ell-1}}$ and $\boldsymbol{b}^{(\ell)} \in \mathbb{R}^{p_\ell}$ for $\ell = 1, \ldots, L$. Their composition $f = f_L \circ f_{L-1} \circ \cdots \circ f_1 : \mathbb{R}^{p_0} \to \mathbb{R}^{p_L}$ is a **single affine map***

$$f(\boldsymbol{x}) = W\boldsymbol{x} + \boldsymbol{b}$$

where the effective weight matrix $W \in \mathcal{M}_{p_L, p_0}$ and effective bias $\boldsymbol{b} \in \mathbb{R}^{p_L}$ are given by

$$W = W^{(L)} W^{(L-1)} \cdots W^{(1)}, \qquad \boldsymbol{b} = \sum_{\ell=1}^{L-1} \left(W^{(L)} \cdots W^{(\ell+1)} \right) \boldsymbol{b}^{(\ell)} + \boldsymbol{b}^{(L)}$$

In batch matrix form on $X \in \mathcal{M}_{n, p_0}$, the L-layer linear network collapses to

$$A^{(L)} = XW^T + \mathbb{1}_n \boldsymbol{b}^T$$

Proof of Theorem 185 (collapse of composed affine maps)

We proceed by induction on the number of layers $L \geq 1$

Base case ($L = 2$): For two layers, substituting $\boldsymbol{a}^{(1)} = W^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}$ into layer 2 gives

$$\begin{aligned}\boldsymbol{a}^{(2)} &= W^{(2)}\boldsymbol{a}^{(1)} + \boldsymbol{b}^{(2)} \\ &= W^{(2)}\big(W^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}\big) + \boldsymbol{b}^{(2)} \\ &= \big(W^{(2)}W^{(1)}\big)\boldsymbol{x} + \big(W^{(2)}\boldsymbol{b}^{(1)} + \boldsymbol{b}^{(2)}\big)\end{aligned}$$

which is a single affine map with $W = W^{(2)}W^{(1)}$ and $\boldsymbol{b} = W^{(2)}\boldsymbol{b}^{(1)} + \boldsymbol{b}^{(2)}$

Inductive step: Suppose after $L-1$ layers that $\boldsymbol{a}^{(L-1)} = \widetilde{W}\boldsymbol{x} + \tilde{\boldsymbol{b}}$ with $\widetilde{W} = W^{(L-1)} \cdots W^{(1)}$. Applying layer L yields

$$\boldsymbol{a}^{(L)} = W^{(L)}\big(\widetilde{W}\boldsymbol{x} + \tilde{\boldsymbol{b}}\big) + \boldsymbol{b}^{(L)} = \big(W^{(L)}\widetilde{W}\big)\boldsymbol{x} + \big(W^{(L)}\tilde{\boldsymbol{b}} + \boldsymbol{b}^{(L)}\big) = W\boldsymbol{x} + \boldsymbol{b} \quad \square$$

Rank bottleneck bound for deep linear networks

Not only does a deep linear network collapse into a single matrix $W = W^{(L)} \cdots W^{(1)}$, the rank of W is choked by the narrowest layer in the network

Theorem 186 (Rank bottleneck bound)

Let $W = W^{(L)} W^{(L-1)} \cdots W^{(1)} \in \mathcal{M}_{p_L, p_0}$ *with* $W^{(\ell)} \in \mathcal{M}_{p_\ell, p_{\ell-1}}$ *for* $\ell = 1, \ldots, L$. *Then*

$$\operatorname{rank}(W) \leq \min_{1 \leq \ell \leq L} \operatorname{rank}\big(W^{(\ell)}\big) \leq \min_{0 \leq \ell \leq L} p_\ell$$

Before proving Theorem 186, recall the fundamental definitions for $M \in \mathcal{M}_{np}$:

$$\operatorname{range}(M) = \big\{M\boldsymbol{x} : \boldsymbol{x} \in \mathbb{R}^p\big\} \subseteq \mathbb{R}^n, \qquad \operatorname{null}(M) = \big\{\boldsymbol{x} \in \mathbb{R}^p : M\boldsymbol{x} = \boldsymbol{0}\big\} \subseteq \mathbb{R}^p$$

and the **rank-nullity theorem**:
$\operatorname{rank}(M) + \operatorname{nullity}(M) = \dim \operatorname{range}(M) + \dim \operatorname{null}(M) = p$

Proof of Theorem 186 – part 1: subspace inclusions

Let $A \in \mathcal{M}_{np}$ and $B \in \mathcal{M}_{pq}$ so that $AB \in \mathcal{M}_{nq}$. We prove two fundamental inclusions:

1. **Range inclusion** $\text{range}(AB) \subseteq \text{range}(A)$: Let $\boldsymbol{y} \in \text{range}(AB)$. Then there exists $\boldsymbol{x} \in \mathbb{R}^q$ such that

$$\boldsymbol{y} = (AB)\boldsymbol{x} = A(B\boldsymbol{x})$$

Setting $\boldsymbol{u} = B\boldsymbol{x} \in \mathbb{R}^p$, we have $\boldsymbol{y} = A\boldsymbol{u} \in \text{range}(A)$. Thus $\text{range}(AB) \subseteq \text{range}(A)$, which implies

$$\text{rank}(AB) = \dim \text{range}(AB) \leq \dim \text{range}(A) = \text{rank}(A)$$

2. **Nullspace inclusion** $\text{null}(B) \subseteq \text{null}(AB)$: Let $\boldsymbol{x} \in \text{null}(B)$, so $B\boldsymbol{x} = \boldsymbol{0}$. Multiplying on the left by A gives

$$(AB)\boldsymbol{x} = A(B\boldsymbol{x}) = A\boldsymbol{0} = \boldsymbol{0} \implies \boldsymbol{x} \in \text{null}(AB)$$

Thus $\text{null}(B) \subseteq \text{null}(AB)$, so $\text{nullity}(B) \leq \text{nullity}(AB)$

Proof of Theorem 186 – part 2: rank of a matrix product

3. **Combining with the rank-nullity theorem**: Since $B \in \mathcal{M}_{pq}$ and $AB \in \mathcal{M}_{nq}$ both have q columns, the rank-nullity theorem gives

$$\text{rank}(AB) = q - \text{nullity}(AB) \leq q - \text{nullity}(B) = \text{rank}(B)$$

Combining parts 1 and 2 yields the fundamental inequality for any compatible matrices A and B:

$$\text{rank}(AB) \leq \min\big(\text{rank}(A), \text{rank}(B)\big)$$

4. **Induction over L layers**: Applying $\text{rank}(AB) \leq \min(\text{rank}(A), \text{rank}(B))$ inductively to $W = W^{(L)} W^{(L-1)} \cdots W^{(1)}$ gives

$$\text{rank}(W) \leq \min_{1 \leq \ell \leq L} \text{rank}\big(W^{(\ell)}\big)$$

Finally, since $W^{(\ell)} \in \mathcal{M}_{p_\ell, p_{\ell-1}}$, its rank cannot exceed its number of rows p_ℓ or columns $p_{\ell-1}$, so $\text{rank}(W^{(\ell)}) \leq \min(p_\ell, p_{\ell-1})$, which gives $\text{rank}(W) \leq \min_{0 \leq \ell \leq L} p_\ell$ □

Consequences of linear collapse and the rank bottleneck

Theorem 185 and Theorem 186 reveal two critical design principles of neural networks:

1. **Depth without non-linearity is futile**: A 100-layer linear network cannot represent any function that a single 1-layer affine map $f(\boldsymbol{x}) = W\boldsymbol{x} + \boldsymbol{b}$ cannot already represent. In particular, a 100-layer linear network **still cannot solve XOR**!
2. **Narrow hidden layers create permanent rank bottlenecks**: Suppose a 3-layer linear network has architecture $\mathbb{R}^{10} \xrightarrow{W^{(1)}} \mathbb{R}^{2} \xrightarrow{W^{(2)}} \mathbb{R}^{50} \xrightarrow{W^{(3)}} \mathbb{R}^{10}$. Even though layer 2 expands to 50 neurons and the output has dimension 10,

$$\operatorname{rank}\left(W^{(3)}W^{(2)}W^{(1)}\right) \leq \min(10, 2, 50, 10) = 2$$

Every output vector $f(\boldsymbol{x})$ is confined to at most a 2-dimensional affine subspace of $\mathbb{R}^{10}$!

Non-linear activations σ_ℓ bend the coordinate hyperplanes at each layer, allowing the network to construct piecewise-linear (for ReLU) or smooth non-linear decision boundaries

Example by hand: collapsing a 2-layer linear network

Problem: Consider a 2-layer linear network mapping $\mathbb{R}^3 \to \mathbb{R}^2 \to \mathbb{R}^3$ ($\sigma_1 = \sigma_2 = \mathrm{id}$) with

$$W^{(1)} = \begin{pmatrix} 1 & 2 & -1 \\ 0 & 1 & 1 \end{pmatrix}, \quad \boldsymbol{b}^{(1)} = \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \quad W^{(2)} = \begin{pmatrix} 2 & 1 \\ -1 & 0 \\ 1 & 1 \end{pmatrix}, \quad \boldsymbol{b}^{(2)} = \begin{pmatrix} 0 \\ 2 \\ 1 \end{pmatrix}$$

Find the collapsed affine map $f(\boldsymbol{x}) = W\boldsymbol{x} + \boldsymbol{b}$ and determine $\mathrm{rank}(W)$

Solution: By Theorem 185,

$$W = W^{(2)}W^{(1)} = \begin{pmatrix} 2 & 1 \\ -1 & 0 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 2 & -1 \\ 0 & 1 & 1 \end{pmatrix} = \begin{pmatrix} 2 & 5 & -1 \\ -1 & -2 & 1 \\ 1 & 3 & 0 \end{pmatrix}, \quad \boldsymbol{b} = W^{(2)}\boldsymbol{b}^{(1)} + \boldsymbol{b}^{(2)} =$$

Example by hand: verifying the rank bottleneck

Although $W \in \mathcal{M}_3$ is a 3×3 matrix, Theorem 186 guarantees that because the hidden layer has width $p_1 = 2$,

$$\text{rank}(W) \leq \min(3, 2, 3) = 2 < 3$$

so W **cannot be invertible** ($\det(W) = 0$)

Indeed, inspecting the rows R_1, R_2, R_3 of $W = \begin{pmatrix} 2 & 5 & -1 \\ -1 & -2 & 1 \\ 1 & 3 & 0 \end{pmatrix}$, we see that

$$R_1 + R_2 = (2 - 1,\ 5 - 2,\ -1 + 1) = (1, 3, 0) = R_3$$

Since R_1 and R_2 are not scalar multiples of each other, $\text{rank}(W) = 2$

How ReLU breaks the rank bound: In our XOR example, the pre-activation matrix $Z^{(1)} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix}$ had column 2 equal to (column 1) $- \mathbb{1}_4$, so no linear combination with $\mathbb{1}_4$ could yield $\boldsymbol{y} = (0, 1, 1, 0)^T$. Applying ReLU zeroed out only $\begin{pmatrix} 0 & 0 \end{pmatrix}$

From single neurons to affine matrix layers

Why non-linearity is essential: collapsing linear networks

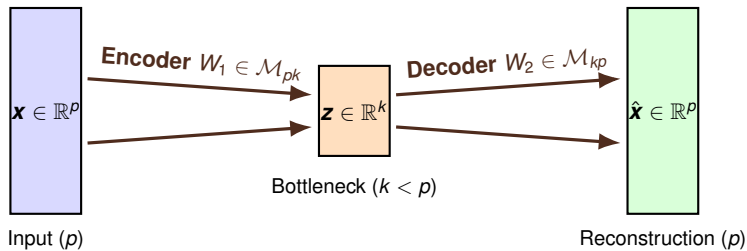
Linear autoencoders and subspace projections (bridge to PCA)

Graph convolutional networks: combining left and right matrix multiplication

When a rank bottleneck is a feature, not a bug!

In supervised classification, an unintended rank bottleneck restricts the expressive power of a linear network

In **unsupervised representation learning** and **dimensionality reduction** (Units 04 and 05), however, we **deliberately** introduce a narrow hidden layer of width $k < p$ to force the network to compress p-dimensional data into k latent coordinates!



Linear autoencoders

Definition 187 (Linear autoencoder)

*Let $\widetilde{X} = C_n X \in \mathcal{M}_{np}$ be a centred data matrix (n observations in $\mathbb{R}^p$, column means zero) and let $1 \leq k < p$ be the bottleneck dimension. A **linear autoencoder** consists of:*

1. *An **encoder matrix** $W_1 \in \mathcal{M}_{pk}$ mapping data to a low-dimensional **latent code***
$$Z = \widetilde{X} W_1 \in \mathcal{M}_{nk}$$
2. *A **decoder matrix** $W_2 \in \mathcal{M}_{kp}$ mapping the latent code back to a reconstruction*
$$\hat{X} = Z W_2 = \widetilde{X} W_1 W_2 \in \mathcal{M}_{np}$$

The autoencoder is trained to minimise the squared Frobenius reconstruction error:

$$\min_{W_1 \in \mathcal{M}_{pk},\ W_2 \in \mathcal{M}_{kp}} \left\| \widetilde{X} - \widetilde{X} W_1 W_2 \right\|_F^2 = \min_{W_1 \in \mathcal{M}_{pk},\ W_2 \in \mathcal{M}_{kp}} \sum_{i=1}^{n} \left\| \tilde{\boldsymbol{x}}_i - W_2^T W_1^T \tilde{\boldsymbol{x}}_i \right\|_2^2$$

Bridge to SVD (Unit 04) and PCA (Unit 05)

Let us connect the linear autoencoder problem directly to the Eckart-Young theorem (Unit 04) and principal component analysis (Unit 05)

- Recall that the centred data matrix $\widetilde{X} \in \mathcal{M}_{np}$ has singular value decomposition

$$\widetilde{X} = U\Sigma V^T = \sum_{j=1}^{r} \sigma_j \boldsymbol{u}_j \boldsymbol{v}_j^T, \qquad \sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$$

  where the right singular vectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_p \in \mathbb{R}^p$ are the orthonormal eigenvectors of the sample covariance matrix $S_X = \frac{1}{n-1}\widetilde{X}^T\widetilde{X}$ (*i.e.*, the **principal axes**)

- Let $V_k = [\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k] \in \mathcal{M}_{pk}$ collect the top k principal axes. In Unit 05, the rank-k PCA reconstruction of $\widetilde{X}$ was shown to be

$$\widetilde{X}_k = U_k \Sigma_k V_k^T = (\widetilde{X} V_k) V_k^T = \widetilde{X}(V_k V_k^T)$$

  where $P_k = V_k V_k^T \in \mathcal{M}_p$ is the **orthogonal projector** onto $\text{span}\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$

Optimal linear autoencoders recover the PCA subspace

Theorem 188 (Baldi and Hornik, 1989 – Eckart-Young corollary)

Let $\widetilde{X} \in \mathcal{M}_{np}$ have singular values $\sigma_1 \geq \cdots \geq \sigma_p \geq 0$ and right singular vectors $V = [\boldsymbol{v}_1, \ldots, \boldsymbol{v}_p] \in \mathcal{M}_p$. For any $k < \text{rank}(\widetilde{X})$, consider

$$\min_{W_1 \in \mathcal{M}_{pk},\, W_2 \in \mathcal{M}_{kp}} \|\widetilde{X} - \widetilde{X} W_1 W_2\|_F^2$$

1. *The minimum reconstruction error is* $\displaystyle\sum_{j=k+1}^{p} \sigma_j^2$
2. *When $\sigma_k > \sigma_{k+1}$, the global minimum on* $\text{range}(\widetilde{X}^T)$ *is achieved when*

$$P = W_1 W_2 = V_k V_k^T$$

is the orthogonal projection matrix onto the top-k principal subspace $\text{span}\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$*, achieved by*

Proof of Theorem 188 (Baldi-Hornik / Eckart-Young corollary)

1. **Lower bound via the rank bottleneck bound and Eckart-Young**: For any $W_1 \in \mathcal{M}_{pk}$ and $W_2 \in \mathcal{M}_{kp}$, Theorem 186 implies

$$\text{rank}(W_1 W_2) \leq \min(p, k, p) = k \implies \text{rank}(\widetilde{X} W_1 W_2) \leq \text{rank}(W_1 W_2) \leq k$$

Hence $\hat{X} = \widetilde{X} W_1 W_2 \in \mathcal{M}_{np}$ is a matrix of rank at most k. By the Eckart-Young theorem (Unit 04), the best rank-k approximation to $\widetilde{X}$ in the Frobenius norm is the truncated SVD $\widetilde{X}_k = U_k \Sigma_k V_k^T$, so for all W_1, W_2:

$$\|\widetilde{X} - \widetilde{X} W_1 W_2\|_F^2 \geq \|\widetilde{X} - \widetilde{X}_k\|_F^2 = \sum_{j=k+1}^{p} \sigma_j^2$$

2. **Attaining the bound**: Let $C \in \text{GL}_k(\mathbb{R})$ be any invertible $k \times k$ matrix and set $W_1 = V_k C \in \mathcal{M}_{pk}$ and $W_2 = C^{-1} V_k^T \in \mathcal{M}_{kp}$. Then

$$W_1 W_2 = (V_k C)(C^{-1} V_k^T) = V_k (C C^{-1}) V_k^T = V_k \mathbb{I}_k V_k^T = V_k V_k^T$$

Since $\widetilde{X} V_k = U \Sigma V^T V_k = U_k \Sigma_k$, we obtain
$\widetilde{X} W_1 W_2 = \widetilde{X} V_k V_k^T = U_k \Sigma_k V_k^T = \widetilde{X}_k$, achieving the exact Eckart-Young minimum $\|\widetilde{X} - \widetilde{X}_k\|_F^2$.

Subspace equivalence vs basis non-uniqueness

Notice a subtle and important distinction in Theorem 188:

- **What is unique**: The product $P = W_1 W_2 = V_k V_k^T \in \mathcal{M}_p$ is the unique orthogonal projector onto the k-dimensional principal subspace $\mathcal{S}_k = \text{span}\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ (satisfying $P^2 = P$ and $P^T = P$)
- **What is not unique (gauge freedom)**: Because $W_1 = V_k C$ and $W_2 = C^{-1} V_k^T$ give the exact same product $W_1 W_2 = V_k V_k^T$ for **any** invertible $C \in \text{GL}_k(\mathbb{R})$, a linear autoencoder without extra constraints learns a basis for the principal subspace $\mathcal{S}_k$, not necessarily the orthogonal principal axes V_k themselves
- **PCA corresponds to choosing** $C = \mathbb{I}_k$: Setting $W_1 = V_k$ and $W_2 = V_k^T$ imposes the orthogonality constraint $W_1^T W_1 = \mathbb{I}_k$ and ties the decoder weights to the transpose of the encoder ($W_2 = W_1^T$)

Orthogonal projector properties and Pythagorean error decomposition

Let us verify that $P_k = V_k V_k^T \in \mathcal{M}_p$ is an **orthogonal projection matrix**:

- **Symmetric**: $P_k^T = \left(V_k V_k^T\right)^T = \left(V_k^T\right)^T V_k^T = V_k V_k^T = P_k$
- **Idempotent**: Since the columns of V_k are orthonormal ($V_k^T V_k = \mathbb{I}_k$),

$$P_k^2 = \left(V_k V_k^T\right)\left(V_k V_k^T\right) = V_k \left(V_k^T V_k\right) V_k^T = V_k \mathbb{I}_k V_k^T = P_k$$

Every centred data matrix $\widetilde{X} \in \mathcal{M}_{np}$ decomposes into two mutually orthogonal components:

$$\widetilde{X} = \underbrace{\widetilde{X} P_k}_{\text{projection onto } \mathcal{S}_k} + \underbrace{\widetilde{X}(\mathbb{I}_p - P_k)}_{\text{residual in } \mathcal{S}_k^{\perp}}$$

Because $P_k(\mathbb{I}_p - P_k) = P_k - P_k^2 = 0$, the squared Frobenius norm splits by Pythagoras:

$$\|\widetilde{X} - \widetilde{X} P_k\|_F^2 = \|\widetilde{X}\|_F^2 - \|\widetilde{X} P_k\|_F^2 = \sum_{j=1}^{p} \sigma_j^2 - \sum_{j=1}^{k} \sigma_j^2 = \sum_{j=k+1}^{p} \sigma_j^2$$

Example by hand: rank-1 linear autoencoder vs PCA

Consider the centred 4×2 data matrix $\widetilde{X} \in \mathcal{M}_{42}$ and bottleneck dimension $k = 1$:

$$\widetilde{X} = \begin{pmatrix} 3 & 4 \\ -3 & -4 \\ 3 & 4 \\ -3 & -4 \end{pmatrix}, \qquad \widetilde{X}^T\widetilde{X} = \begin{pmatrix} 36 & 48 \\ 48 & 64 \end{pmatrix} = 100 \begin{pmatrix} 0.36 & 0.48 \\ 0.48 & 0.64 \end{pmatrix}$$

- The unit eigenvector of $\widetilde{X}^T\widetilde{X}$ for $\lambda_1 = \sigma_1^2 = 100$ is $\boldsymbol{v}_1 = \begin{pmatrix} 0.6 \\ 0.8 \end{pmatrix}$ (and $\sigma_2^2 = 0$ for $\boldsymbol{v}_2 = \begin{pmatrix} -0.8 \\ 0.6 \end{pmatrix}$)
- The PCA choice ($C = 1$) uses $W_1 = \boldsymbol{v}_1 = \begin{pmatrix} 0.6 \\ 0.8 \end{pmatrix}$ and $W_2 = \boldsymbol{v}_1^T = \begin{pmatrix} 0.6 & 0.8 \end{pmatrix}$
- Any scaled pair $\widetilde{W}_1 = c\boldsymbol{v}_1 = \begin{pmatrix} 1.2 \\ 1.6 \end{pmatrix}$ and $\widetilde{W}_2 = c^{-1}\boldsymbol{v}_1^T = \begin{pmatrix} 0.3 & 0.4 \end{pmatrix}$ (with $c = 2$) gives the **exact same orthogonal projector**:

$$P = W_1 W_2 = \widetilde{W}_1 \widetilde{W}_2 = \boldsymbol{v}_1 \boldsymbol{v}_1^T = \begin{pmatrix} 0.36 & 0.48 \\ 0.48 & 0.64 \end{pmatrix}, \qquad \hat{X} = \widetilde{X}P = \widetilde{X}$$

R demonstration: training a linear autoencoder recovers PCA

Generate a centred 3D dataset ($n = 150, p = 3$) and train a linear autoencoder ($k = 2$) by minimising $\|\widetilde{X} - \widetilde{X} W_1 W_2\|_F^2$ from random initial weights:

```
set.seed(2740); n = 150; p = 3; k = 2
M_mix = matrix(c(3, 2, 1, 0, 1.5, 1, 0, 0, 0.3), nrow = 3, byrow = TRUE)
X_tilde = scale(matrix(rnorm(n * p), n, p) %*% M_mix, center = TRUE, scale =
V_k = svd(X_tilde)$v[, 1:k];  P_pca = V_k %*% t(V_k)

ae_loss = function(th) {
  W1 = matrix(th[1:6], p, k);  W2 = matrix(th[7:12], k, p)
  sum((X_tilde - X_tilde %*% W1 %*% W2)^2)
}
opt = optim(rnorm(12, sd = 0.5), ae_loss, method = "BFGS",
            control = list(maxit = 500, reltol = 1e-12))
W1_hat = matrix(opt$par[1:6], p, k);  W2_hat = matrix(opt$par[7:12], k, p)
P_ae = W1_hat %*% W2_hat
```

Comparing the learned projector $W_1 W_2$ with $V_k V_k^T$

Even though W_1 and V_k differ by an invertible change of basis $C \in \mathrm{GL}_2(\mathbb{R})$, their product $W_1 W_2$ matches the orthogonal PCA projector $V_k V_k^T$ to high precision:

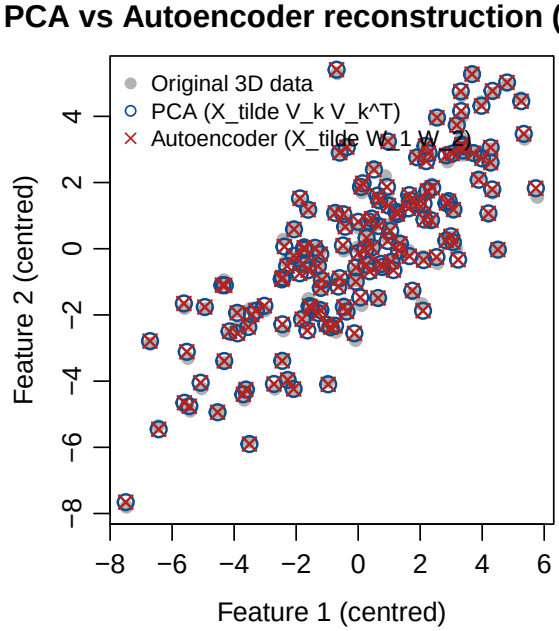
```
round(P_pca, 4)
```

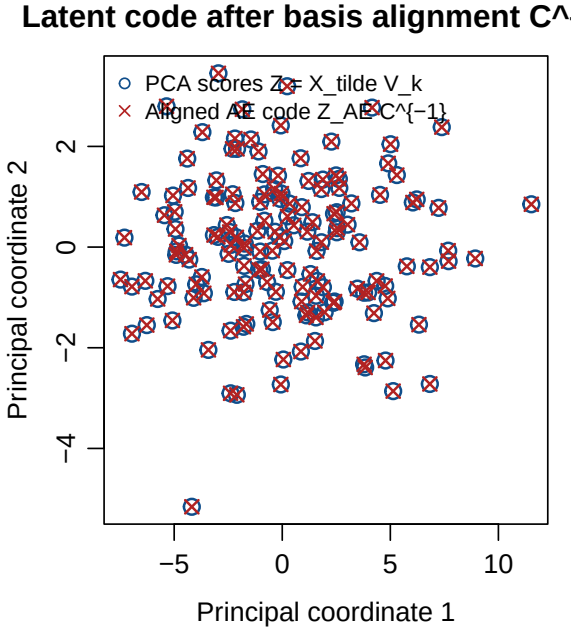
```
##         [,1]   [,2]    [,3]
## [1,]  0.9897 0.0570 -0.0832
## [2,]  0.0570 0.6838  0.4615
## [3,] -0.0832 0.4615  0.3265
```

```
round(P_ae, 4)
```

```
##         [,1]   [,2]    [,3]
## [1,]  0.9897 0.0570 -0.0832
## [2,]  0.0570 0.6838  0.4615
## [3,] -0.0832 0.4615  0.3265
```

```
max(abs(P_pca - P_ae))
```


PCA vs Autoencoder reconstruction (x
Original 3D data
PCA (X_tilde V_k V_k^T)
Autoencoder (X_tilde W_1 W_2)
Feature 2 (centred)
Feature 1 (centred)
−8
−6
−4
−2
0
2
4
6


Latent code after basis alignment C^{
PCA scores Z = X_tilde V_k
Aligned AE code Z_AE C^{−1}
Principal coordinate 2
Principal coordinate 1
−5
0
5
10

From single neurons to affine matrix layers

Why non-linearity is essential: collapsing linear networks

Linear autoencoders and subspace projections (bridge to PCA)

Graph convolutional networks: combining left and right matrix multiplication

Graph convolutional networks: combining left and right matrix multiplication

Left-multiplication vs right-multiplication on X

Self-loops, degree normalisation and the GCN layer

What when the rows of X are connected by a graph?

In a standard MLP or autoencoder, the rows of $X \in \mathcal{M}_{np}$ are treated as independent observations

In many applications (Units 08 to 10), however, the n rows of $X \in \mathcal{M}_{np}$ are the **vertices of a graph** $G = (V, E)$ with adjacency matrix $A \in \mathcal{M}_n$ (in graph machine learning, the data matrix X is called the **vertex feature matrix** or **node feature matrix**):

- **Citation networks**: vertices are scientific papers, rows $\boldsymbol{x}_i^T \in \mathbb{R}^p$ are word-frequency vectors and edges are citations
- **Social and epidemiological networks**: vertices are individuals or cities, features are demographic variables and edges are contact or mobility links
- **Molecular graphs**: vertices are atoms, features are atomic properties and edges are chemical bonds

How can a neural network layer use **both** the vertex features $X \in \mathcal{M}_{np}$ and the graph structure $A \in \mathcal{M}_n$?

The fundamental linear algebra duality: left vs right multiplication

Given a data matrix (vertex feature matrix) $X \in \mathcal{M}_{np}$ (n vertices in rows, p features in columns), consider what happens when we multiply X on the **right** versus on the **left**:

Property 189 (Right-multiplication mixes features; left-multiplication mixes vertices)

Let $X \in \mathcal{M}_{np}$ have rows $\boldsymbol{x}_1^T, \ldots, \boldsymbol{x}_n^T \in \mathbb{R}^{1\times p}$ and columns $\boldsymbol{c}_1, \ldots, \boldsymbol{c}_p \in \mathbb{R}^n$:

1. ***Right-multiplication by a weight matrix** $W \in \mathcal{M}_{pq}$:*

$$(XW)_{i\bullet} = \boldsymbol{x}_i^T W \in \mathbb{R}^{1\times q} \qquad (i = 1, \ldots, n)$$

   *Each vertex i transforms its **own features** independently of all other vertices (this is a standard MLP layer, completely ignoring the graph G)*

2. ***Left-multiplication by a graph operator** $S \in \mathcal{M}_n$:*

$$(SX)_{i\bullet} = \sum_{j=1}^{n} s_{ij} \boldsymbol{x}_j^T \in \mathbb{R}^{1\times p} \qquad (i = 1, \ldots, n)$$

Visualising the two-sided sandwich product SXW

Combining left-multiplication by a graph operator $S \in \mathcal{M}_n$ and right-multiplication by a feature weight matrix $W \in \mathcal{M}_{pq}$ produces the **sandwich product** $SXW \in \mathcal{M}_{nq}$:



By associativity of matrix multiplication, $(SX)W = S(XW)$:

- Viewing it as $(SX)W$: first **aggregate/smooth features** across graph neighbours ($SX \in \mathcal{M}_{np}$), then **project features** using trainable weights W
- Viewing it as $S(XW)$: first **project each vertex's features** into $\mathbb{R}^q$ ($XW \in \mathcal{M}_{nq}$), then **aggregate the projected vectors** across graph neighbours via S

Graph convolutional networks: combining left and right matrix multiplication

Left-multiplication vs right-multiplication on X

Self-loops, degree normalisation and the GCN layer

Why raw adjacency multiplication AX has two flaws

Suppose we set $S = A$, where $A = [a_{ij}] \in \mathcal{M}_n$ is the unweighted adjacency matrix of an undirected simple graph G ($a_{ij} = 1$ if $i \sim j$ and 0 otherwise). Then row i of AX is

$$(AX)_{i\bullet} = \sum_{j=1}^{n} a_{ij} \boldsymbol{x}_j^T = \sum_{j \in \mathcal{N}(i)} \boldsymbol{x}_j^T$$

where $\mathcal{N}(i) = \{j \in V : i \sim j\}$ is the neighbourhood of vertex i

Using raw A as our left-multiplier has **two structural flaws**:

1. **Forgetting a vertex's own features**: Because a simple graph has no self-loops ($a_{ii} = 0$), $(AX)_{i\bullet}$ sums the neighbours' features $\boldsymbol{x}_j^T$ ($j \in \mathcal{N}(i)$) but **completely discards** vertex i's own feature vector $\boldsymbol{x}_i^T$!
2. **Exploding/vanishing scales across vertex degrees**: A high-degree hub with $d_i = 1000$ sums 1000 vectors, while a leaf vertex with $d_i = 1$ takes only 1 vector. Across multiple layers, feature magnitudes explode at hubs!

Fix 1 and Fix 2: self-loops and degree normalisation

We fix both flaws using standard graph matrix operations from Units 08 to 10:

1. **Fix 1 – Add self-loops to every vertex**: Define the **augmented adjacency matrix**

$$\tilde{A} = A + \mathbb{I}_n \in \mathcal{M}_n$$

   Now $\tilde{a}_{ii} = 1$, so $(\tilde{A}X)_{i\bullet} = \boldsymbol{x}_i^T + \sum_{j \in \mathcal{N}(i)} \boldsymbol{x}_j^T$ includes vertex i itself

2. **Fix 2 – Normalise by the augmented degree matrix**: Let $D = \text{diag}(d_1, \ldots, d_n)$ be the degree matrix of G, where $d_i = \sum_{j=1}^n a_{ij} = |\mathcal{N}(i)|$. The degree matrix of $\tilde{A}$ is the **augmented degree matrix**

$$\tilde{D} = D + \mathbb{I}_n = \text{diag}(d_1 + 1, d_2 + 1, \ldots, d_n + 1) \in \mathcal{M}_n$$

   Since $d_i \geq 0$, $\tilde{d}_{ii} = d_i + 1 \geq 1 > 0$, so $\tilde{D}$ is **always invertible**, even if G has isolated vertices!

Two canonical normalised graph smoothing operators

Using $\tilde{A} = A + \mathbb{I}_n$ and $\tilde{D} = D + \mathbb{I}_n$, there are two standard choices for the smoothing operator $S \in \mathcal{M}_n$:

Definition 190 (Random-walk and symmetric normalised graph operators)

1. **Random-walk (row-normalised) propagation matrix**:

$$S_{\text{rw}} = \tilde{D}^{-1}\tilde{A}, \qquad \left(S_{\text{rw}}\right)_{ij} = \frac{\tilde{a}_{ij}}{d_i + 1}$$

Here S_{rw} is **row-stochastic** ($S_{\text{rw}}\mathbb{1}_n = \mathbb{1}_n$) and row i of $S_{\text{rw}}X$ is the **exact arithmetic average** of vertex i and its neighbours:

$$\left(\tilde{D}^{-1}\tilde{A}X\right)_{i\bullet} = \frac{1}{d_i + 1}\boldsymbol{x}_i^T + \sum_{j \in \mathcal{N}(i)} \frac{1}{d_i + 1}\boldsymbol{x}_j^T$$

2. **Symmetric normalised propagation matrix** (Kipf and Welling, 2017):

The graph convolutional network (GCN) layer

Definition 191 (Graph convolutional network layer)

Given a graph $G = (V, E)$ with augmented adjacency matrix $\tilde{A} = A + \mathbb{I}_n$, augmented degree matrix $\tilde{D} = D + \mathbb{I}_n$ and propagation operator $S \in \{\tilde{D}^{-1}\tilde{A},\ \tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}\}$, an L-layer ***Graph Convolutional Network (GCN)*** *with layer widths $p_0, p_1, \ldots, p_L$ sets $H^{(0)} = X \in \mathcal{M}_{n,p_0}$ (where $p_0 = p$) and updates vertex representations at layer $\ell \in \{1, \ldots, L\}$ via*

$$H^{(\ell)} = \sigma_\ell\big(SH^{(\ell-1)}W^{(\ell)}\big) \in \mathcal{M}_{n,p_\ell}$$

where $W^{(\ell)} \in \mathcal{M}_{p_{\ell-1},p_\ell}$ is the trainable feature weight matrix of layer ℓ and σ_ℓ is a component-wise activation function

Notice that if the graph has **no edges** ($A = 0$), then $\tilde{A} = \tilde{D} = \mathbb{I}_n$, so $S = \mathbb{I}_n$ and the GCN layer reduces to a standard MLP layer $H^{(\ell)} = \sigma_\ell\big(H^{(\ell-1)}W^{(\ell)}\big)$!

Multi-hop aggregation and over-smoothing (link to Markov chains)

What happens as we stack L GCN layers?

- In layer 1, vertex i averages features from its 1-hop neighbourhood $\mathcal{N}(i) \cup \{i\}$
- In layer 2, vertex i averages layer-1 features from its neighbours, which already contain information from *their* neighbours – aggregating the 2-**hop neighbourhood**!
- In a linear GCN ($\sigma_\ell = \mathrm{id}$), k layers compute $H^{(k)} = S_{\mathrm{rw}}^k X W^{(1)} \dots W^{(k)} = \left(\tilde{D}^{-1}\tilde{A}\right)^k XW$

Connection to Unit 08 (Markov chains) – the over-smoothing phenomenon:

- Because $S_{\mathrm{rw}} = \tilde{D}^{-1}\tilde{A}$ is the transition matrix of a regular Markov chain on a connected graph (self-loops make it aperiodic!), as $k \to \infty$,

$$\lim_{k\to\infty} \left(\tilde{D}^{-1}\tilde{A}\right)^k = \mathbb{1}_n \pi^T \quad \text{(a rank-1 matrix)}, \qquad \pi_j = \frac{d_j + 1}{\sum_{m=1}^{n}(d_m + 1)}$$

- Thus $\left(\tilde{D}^{-1}\tilde{A}\right)^k X \to \mathbb{1}_n(\pi^T X)$: every row becomes **identical**, washing out all local differences between vertices! Hence GCNs in practice are kept shallow ($L = 2$ or 3)

Brief remark: Transformers and self-attention as dynamic graph operators

In large language models (Transformers), a sequence of n tokens is represented by a data matrix (token embedding matrix) $X \in \mathcal{M}_{np}$. A **self-attention head** projects X using three weight matrices $W_Q, W_K \in \mathcal{M}_{p,d_k}$ and $W_V \in \mathcal{M}_{p,d_v}$:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V$$

and computes the output matrix

$$\text{Attention}(X) = \underbrace{\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)}_{S(X)\in\mathcal{M}_n} XW_V$$

where softmax is applied row-wise so every row of $S(X)$ is non-negative and sums to 1

Linear algebra unification: Self-attention is also a sandwich product $S(X)XW_V$! Instead of a fixed graph operator $S = \tilde{D}^{-1}\tilde{A}$, the row-stochastic matrix $S(X) \in \mathcal{M}_n$ is a **learned, data-dependent adjacency matrix** computed dynamically from the pairwise inner products $QK^T = XW_QW_K^TX^T$

Mini-example: self-attention as a row-stochastic left multiplier

Consider a 2-token sequence ("river", "bank") with 2×2 embedding matrix $X = \begin{pmatrix} 0.5 & 0.2 \\ 0.8 & 0.6 \end{pmatrix}$ and $W_Q = W_K = W_V = \mathbb{I}_2$ ($d_k = 2$), so $Q = K = V = X$:

1. **Pairwise dot-product score matrix** $QK^T = XX^T$:

$$\frac{QK^T}{\sqrt{2}} = \frac{1}{\sqrt{2}} \begin{pmatrix} 0.5 & 0.2 \\ 0.8 & 0.6 \end{pmatrix} \begin{pmatrix} 0.5 & 0.8 \\ 0.2 & 0.6 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 0.29 & 0.52 \\ 0.52 & 1.00 \end{pmatrix} \approx \begin{pmatrix} 0.205 & 0.368 \\ 0.368 & 0.707 \end{pmatrix}$$

2. **Row-wise softmax produces a** 2×2 **row-stochastic matrix** $S(X)$:

$$S(X) = \mathrm{softmax}\left(\frac{QK^T}{\sqrt{2}}\right) \approx \begin{pmatrix} 0.46 & 0.54 \\ 0.42 & 0.58 \end{pmatrix}$$

3. **Left-multiplying** $V = X$ **forms convex combinations of token rows**:

$$\boldsymbol{z}_{\text{river}}^T = 0.46 \begin{pmatrix} 0.5 & 0.2 \end{pmatrix} + 0.54 \begin{pmatrix} 0.8 & 0.6 \end{pmatrix} = \begin{pmatrix} 0.662 & 0.416 \end{pmatrix}$$

Example by hand: 1-layer GCN forward pass on a 3-vertex path

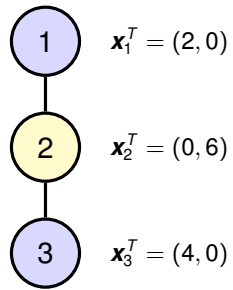
Problem: Consider the 3-vertex path graph $1 - 2 - 3$ with vertex feature matrix $X \in \mathcal{M}_{32}$ and weight matrix $W \in \mathcal{M}_{22}$:

$$X = \begin{pmatrix} 2 & 0 \\ 0 & 6 \\ 4 & 0 \end{pmatrix}, \qquad W = \begin{pmatrix} 2 & -1 \\ -1 & 1 \end{pmatrix}$$

Compute the 1-layer row-normalised ReLU GCN output

$$H = \mathrm{ReLU}\big(\tilde{D}^{-1}\tilde{A}XW\big)$$

by hand



Example by hand: step 1 – graph matrices A, $\tilde{A}$, $\tilde{D}^{-1}$ and S_{rw}

For the path graph $1 - 2 - 3$, the adjacency matrix A and augmented adjacency matrix $\tilde{A} = A + \mathbb{I}_3$ are

$$A = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}, \qquad \tilde{A} = A + \mathbb{I}_3 = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 1 \end{pmatrix}$$

The vertex degrees are $d_1 = 1$, $d_2 = 2$ and $d_3 = 1$, so $\tilde{D} = D + \mathbb{I}_3$, $\tilde{D}^{-1}$ and $S_{\text{rw}} = \tilde{D}^{-1}\tilde{A}$ are

$$\tilde{D} = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 2 \end{pmatrix}, \quad \tilde{D}^{-1} = \begin{pmatrix} \frac{1}{2} & 0 & 0 \\ 0 & \frac{1}{3} & 0 \\ 0 & 0 & \frac{1}{2} \end{pmatrix}, \quad S_{\text{rw}} = \tilde{D}^{-1}\tilde{A} = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ 0 & \frac{1}{2} & \frac{1}{2} \end{pmatrix}$$

Example by hand: step 2 – left-multiplying X and right-multiplying W

Neighbourhood feature averaging (left-multiplication by $\tilde{D}^{-1}\tilde{A}$):

$$\tilde{D}^{-1}\tilde{A}X = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ 0 & \frac{1}{2} & \frac{1}{2} \end{pmatrix} \begin{pmatrix} 2 & 0 \\ 0 & 6 \\ 4 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 3 \\ 2 & 2 \\ 2 & 3 \end{pmatrix}$$

Notice how vertex 2 (which initially had feature $x_{21} = 0$) acquired a positive first feature $(2 + 0 + 4)/3 = 2$ from its neighbours 1 and 3!

Feature projection and ReLU activation (right-multiplication by W, then $\max(0, \cdot)$):

$$Z = \left(\tilde{D}^{-1}\tilde{A}X\right)W = \begin{pmatrix} 1 & 3 \\ 2 & 2 \\ 2 & 3 \end{pmatrix} \begin{pmatrix} 2 & -1 \\ -1 & 1 \end{pmatrix} = \begin{pmatrix} -1 & 2 \\ 2 & 0 \\ 1 & 1 \end{pmatrix}, \qquad H = \mathrm{ReLU}(Z) = \begin{pmatrix} 0 & 2 \\ 2 & 0 \\ 1 & 1 \end{pmatrix}$$

Verification in R: 3-vertex GCN hand calculation

```
A = matrix(c(0, 1, 0,  1, 0, 1,  0, 1, 0), nrow = 3, byrow = TRUE)
A_tilde = A + diag(3)
S_rw = diag(1 / rowSums(A_tilde)) %*% A_tilde
X = matrix(c(2, 0,  0, 6,  4, 0), nrow = 3, byrow = TRUE)
W = matrix(c(2, -1,  -1, 1), nrow = 2, byrow = TRUE)
SX = S_rw %*% X;  Z = SX %*% W;  H = pmax(0, Z)
cbind(SX = SX, Z = Z, H = H)
```

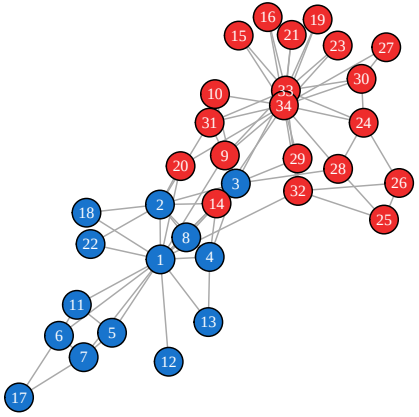
```
##                H
## [1,] 1 3 -1 2 0
## [2,] 2 2  2 0 2
## [3,] 2 3  1 1 1
```

R demonstration: 2-layer GCN on Zachary's Karate Club graph

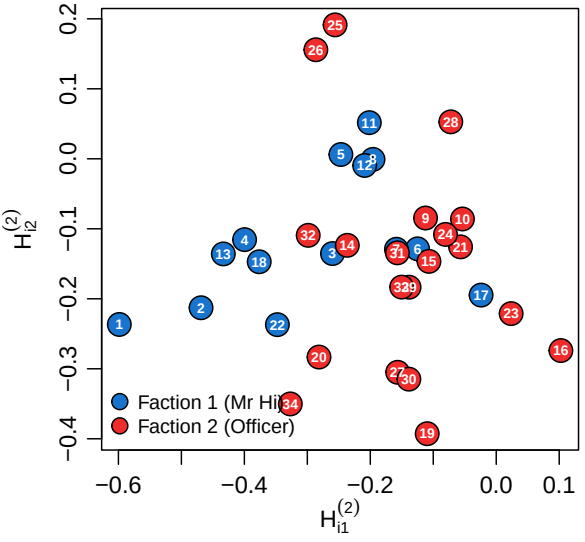
Kipf and Welling (2017) showed on Zachary's Karate Club network ($n = 34$ vertices in 2 factions) that **even with random untrained weights** $W^{(1)}, W^{(2)}$, a 2-layer GCN $H^{(2)} = \tanh\big(S_{\text{sym}}\ \tanh(S_{\text{sym}} X W^{(1)})\ W^{(2)}\big)$ with $X = \mathbb{I}_{34}$ separates the two communities in $\mathbb{R}^2$:

```
G_karate = make_graph("Zachary")
A_k = as.matrix(as_adjacency_matrix(G_karate));  n_k = nrow(A_k)
A_tilde_k = A_k + diag(n_k)
D_inv_sqrt = diag(1 / sqrt(rowSums(A_tilde_k)))
S_sym = D_inv_sqrt %*% A_tilde_k %*% D_inv_sqrt
set.seed(42)
W1_k = matrix(rnorm(n_k * 8, sd = 0.6), n_k, 8)
W2_k = matrix(rnorm(8 * 2, sd = 0.6), 8, 2)
H1_k = tanh(S_sym %*% diag(n_k) %*% W1_k)
H2_k = tanh(S_sym %*% H1_k %*% W2_k)
```

Zachary's Karate Club graph (2 factions)



2−layer GCN embedding in R^2 (untrained

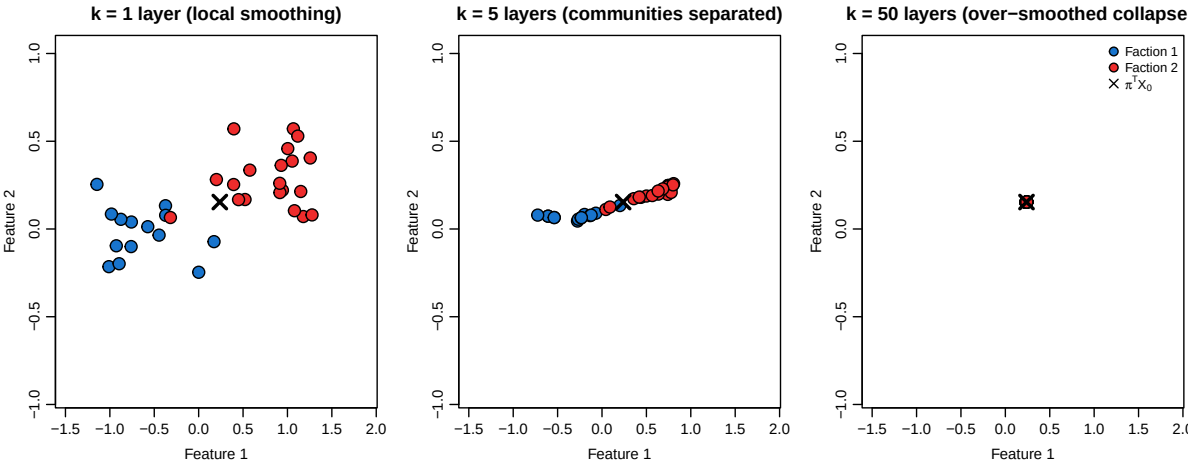


R demonstration: visualising Markov chain over-smoothing as k grows

Applying $S_{\text{rw}} = \tilde{D}^{-1}\tilde{A}$ for $k = 1, 5, 50$ steps to initial 2D features $X_0 \in \mathcal{M}_{34,2}$ converges to the rank-1 stationary average $\mathbb{1}_n(\pi^T X_0)$:

```
S_rw_k = diag(1 / rowSums(A_tilde_k)) %*% A_tilde_k;  set.seed(7)
X0_k = cbind(ifelse(faction == 1, -1, 1) + rnorm(n_k, sd = 0.4),
             rnorm(n_k, sd = 0.6))
mat_pow = function(M, k) Reduce(`%*%`, replicate(k, M, simplify = FALSE))
X_step1  = S_rw_k %*% X0_k;  X_step5 = mat_pow(S_rw_k, 5) %*% X0_k
X_step50 = mat_pow(S_rw_k, 50) %*% X0_k
pi_stat = rowSums(A_tilde_k) / sum(A_tilde_k)
rbind(Row1_k50 = X_step50[1, ], Stat_avg = as.vector(t(pi_stat) %*% X0_k))

##                [,1]      [,2]
## Row1_k50  0.2353291 0.1530728
## Stat_avg  0.2382113 0.1534625
```


k = 1 layer (local smoothing)
k = 5 layers (communities separated)
k = 50 layers (over−smoothed collapse!)
Faction 1
Faction 2
$\pi^T X_0$
Feature 1
Feature 2
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0
−1.0
−0.5
0.0
0.5
1.0

Summary: neural networks through the lens of linear algebra

Architecture	**Layer operator on $X \in \mathcal{M}_{np}$**	**Core linear algebra mechanism**
Single neuron	$\sigma\big(X\boldsymbol{w} + b\mathbb{1}_n\big)$	Affine hyperplane $\boldsymbol{w}^T\boldsymbol{x} + b = 0$ (Unit 06)
MLP layer	$\sigma\big(XW^T + \mathbb{1}_n\boldsymbol{b}^T\big)$	Right-multiplication + rank-1 bias broadcast
Deep linear net	$X\big(W^{(L)}\cdots W^{(1)}\big)^T + \mathbb{1}_n\boldsymbol{b}^T$	Collapses to 1 matrix; $\operatorname{rank}(W) \le \min_\ell p_\ell$
Linear autoencoder	$\hat{X} = \widetilde{X}W_1W_2 \;\; (k < p)$	Optimal $W_1W_2 = V_kV_k^T$ is PCA (Units 04–05)
GCN layer	$\sigma\big(\tilde{D}^{-1}\tilde{A}XW\big)$	Left graph smoothing $\times$ right feature projection
Self-attention	$\operatorname{softmax}\big(\frac{XW_QW_K^TX^T}{\sqrt{d_k}}\big)XW_V$	Data-dependent row-stochastic left multiplier

Modern deep learning architectures are built by composing the exact matrix factorisations, subspace projections and spectral graph operators developed throughout MATH 2740